# 品質管理系統（QMS）

PC 網頁管理後台 + Android 手機檢驗 App + Django REST API + MySQL

# 1. 系統專案目標

建立 PC 與 Android 手機雙端的品質管理系統

PC：負責管理產品、檢驗標準、批次、異常與報表

Android：提供現場檢驗、填寫結果與上傳資料

後端：Django REST API 統一處理資料與權限

資料庫：MySQL 儲存所有系統資料

# 2. 整體系統架構

PC Web 管理後台 → HTTP / REST API → Django → MySQL

Android 手機檢驗 App → HTTP / REST API → Django → MySQL

兩端共用同一套後端與資料庫，避免資料不同步

圖片檔案由 Django Media 管理，資料庫儲存圖片路徑

# 3. 使用者角色與權限

管理員：帳號、人員、產品、系統資料與全部管理

品質主管：查看檢驗結果、處理異常、審核與結案

檢驗人員：查看待檢驗產品、執行檢驗、回報異常

# 4. PC 網頁管理後台

Dashboard：今日檢驗、合格、不良數量與品質趨勢

產品管理：產品分類、產品編號、規格與狀態

檢驗項目管理：標準值、上下限、檢驗方式

檢驗批次管理：建立批次、數量、檢驗狀態

異常與報表：不良品、改善措施、品質統計

# 5. Android 手機檢驗 App

使用者登入與身分驗證

查看待檢驗產品或批次

掃描 QR Code 取得產品資料（進階功能）

逐項填寫檢驗結果與測量數值

不良品拍照、填寫原因並上傳

查看個人檢驗紀錄

# 6. 現場品質檢驗流程

1
登入系統

2
查看待檢驗產品

3
掃描 / 選擇產品

4
開始檢驗

5
填寫結果

6
送出紀錄

7
主管審核

# 7. 品質異常處理流程

發現不良品

建立異常紀錄

品質主管確認

分析問題原因

制定改善措施

執行改善 → 驗證 → 結案

# 8. MySQL 資料庫主要資料表

roles、users：角色與使用者

product_categories、products：產品分類與產品

inspection_items、product_inspection_items：檢驗項目與產品關聯

inspection_batches：檢驗批次

inspection_records、inspection_results：檢驗紀錄與明細

defect_types、defect_records、defect_images：不良品與圖片

corrective_actions：改善措施與處理進度

# 9. 核心資料關聯（ERD 概念）

roles 1 → N users

product_categories 1 → N products

products N ↔ N inspection_items（透過 product_inspection_items）

products 1 → N inspection_batches

inspection_batches 1 → N inspection_records

inspection_records 1 → N inspection_results

inspection_records 1 → N defect_records → defect_images / corrective_actions

# 10. Django 後端模組規劃

accounts：登入、使用者與角色權限

products：產品、分類與檢驗項目

inspections：批次、檢驗紀錄與結果

defects：不良品、圖片與改善措施

dashboard：統計資料與品質分析

API：提供 PC 與 Android 共用 REST API

# 11. REST API 規劃

POST /api/login/：使用者登入

GET /api/inspection/pending/：取得待檢驗產品

GET /api/products/{id}/inspection-items/：取得檢驗項目

POST /api/inspection/submit/：送出檢驗結果

POST /api/defects/：建立不良品紀錄

GET /api/dashboard/：取得品質統計資料

# 12. 建議開發順序

1
資料庫設計

2
Django 模型

3
PC 後台

4
REST API

5
Android App

6
進階功能

# 13. 第一版 MVP 功能

PC：登入、產品管理、檢驗項目、檢驗批次、檢驗結果查看

Android：登入、待檢驗清單、填寫檢驗結果、上傳資料

後端：Django REST API

資料庫：MySQL

先完成完整基本流程，再加入 QR Code、拍照、圖表與通知

# 14. 進階功能規劃

QR Code / Barcode 產品掃描

手機拍照上傳不良品圖片

Dashboard 圖表與不良率統計

品質異常通知

PDF 品質報表匯出

主管審核與改善措施追蹤

# 15. 系統最終目標

PC：提供完整的管理、設定與品質數據分析

Android：提供快速、方便的現場品質檢驗

Django：作為統一後端與 API 服務

MySQL：集中管理所有產品與品質資料

完成後可形成完整的企業品質管理系統專題